

THREAT MODELING

Hasil Vulnerability Assessment Awal

IPB Food Hub & UMKM

Platform Marketplace Digital UMKM Kampus Berbasis Web

Metodologi: STRIDE dan OWASP Top 10 (2021)

Metode: Static Application Security Testing (SAST) Source Code Review

Status: Assessment dengan Implementasi Mitigasi Parsial

Daftar Isi

1. Ringkasan Eksekutif.....	3
2. Konteks Keamanan	4
2.1. Kekhususan Konteks: Data UMKM dan Mahasiswa.....	4
2.2. Aliran Data Utama	4
2.3. Aktor Ancaman	5
3. Analisis STRIDE.....	6
3.1. Metodologi	6
3.2. Analisis per Komponen.....	6
3.2.1. Modul Autentikasi.....	6
3.2.2. Modul Guard dan Otorisasi (RBAC).....	7
3.2.3. Data Sensitif dan Enkripsi AES-256-GCM.....	7
3.2.4. Tanda Tangan Digital RSA-2048 PSS	8
3.2.5. Upload File dan Media (Cloudinary)	9
4. Vulnerability Assessment Awal.....	9
4.1. Temuan Detail.....	9
4.1.1. [VA-01] Tidak Ada Rate Limiting pada Endpoint Login ,TINGGI.....	9
4.1.2. [VA-02] Tidak Ada Mekanisme Account Lockout ,TINGGI.....	10
4.1.3. [VA-03] JWT Stateless ,Tidak Ada Revokasi Token Server-Side ,SEDANG	10
4.1.4. [VA-04] CORS Mengizinkan Semua Method dan Header ,SEDANG	11
4.1.5. [VA-05] Tidak Ada Refresh Token ,SEDANG	11
4.1.6. [VA-06] Validasi MIME Type Upload Gambar Tidak Eksplisit ,SEDANG.....	12
4.1.7. [VA-07] Inkonsistensi Penanganan Error ,RENDAH.....	12
5. Matriks Risiko.....	13
5.1. Prioritas Remediasi	15
6. Rekomendasi Implementasi	16
6.1. Jangka Pendek.....	16
6.1.1. Rate Limiting dengan SlowAPI	16
6.1.2. Account Lockout.....	17
6.2. Jangka Menengah.....	17
6.2.1. Token Revocation via Hash di Database.....	17
6.2.2. CORS Lebih Ketat	18
7. Kesimpulan	18

1. Ringkasan Eksekutif

Dokumen ini menyajikan hasil *threat modeling* dan *vulnerability assessment* awal terhadap sistem IPB Food Hub & UMKM, platform marketplace digital yang mempertemukan mahasiswa IPB sebagai pembeli dengan pelaku UMKM kampus sebagai penjual. Sistem ini memproses data keuangan dan data pribadi yang memerlukan perlindungan ketat:

- Data PII sensitif pengguna: nomor telepon, NIK, nomor rekening, nomor e-wallet (dienkripsi AES-256-GCM)
- Data transaksi keuangan: nominal, status, bukti pembayaran (ditandatangani RSA-2048 PSS)
- Kredensial akun: password di-hash bcrypt; JWT untuk session management
- Log audit: seluruh aktivitas autentikasi dan akses API dicatat

Assessment menemukan 7 temuan kerentanan dengan distribusi sebagai berikut, yakni 2 kategori kritis berupa enkripsi data dan digital signature yang telah dimitigasi sepenuhnya dalam implementasi.

Tabel 1 Ringkasan temuan vulnerability assessment

Tingkat Risiko	Jumlah	Contoh Temuan	Status
Kritis (Dimitigasi)	2	AES-256-GCM enkripsi PII; RSA-2048 PSS tanda tangan transaksi	Sudah dimitigasi
Tinggi	2	Tidak ada rate limiting login; tidak ada account lockout	Belum dimitigasi
Sedang	3	JWT stateless (tanpa revokasi); tidak ada refresh token; CORS allow-all methods	Belum dimitigasi
Rendah	1	Inkonsistensi penanganan error di beberapa endpoint	Belum dimitigasi

Rekomendasi utama: Prioritaskan implementasi *rate limiting* dan *account lockout* pada endpoint login untuk mencegah brute force dan credential stuffing. Selanjutnya, implementasikan mekanisme revokasi token server-side dan refresh token untuk memperkecil *window of exposure* jika token dicuri.

2. Konteks Keamanan

2.1. Kekhususan Konteks: Data UMKM dan Mahasiswa

IPB Food Hub & UMKM memproses dua kategori data yang secara hukum dan operasional memerlukan perlindungan khusus sebagai berikut.

Tabel 2 Kategori data sensitif yang diproses sistem

Kategori Data	Sumber	Sensitivitas
Nomor rekening & e-wallet UMKM	Form registrasi; disimpan di kolom nomor_rekening, nomor_ewallet (AES-GCM)	Sangat Tinggi
NIK pelaku UMKM	Verifikasi identitas; kolom nik (AES-GCM)	Sangat Tinggi
Nomor telepon pengguna	Form registrasi; kolom no_telp (AES-GCM)	Tinggi
Bukti transfer pembayaran (gambar)	Upload mahasiswa; disimpan di Cloudinary; URL di DB	Tinggi
Riwayat pesanan & transaksi	Tabel pesanan, item_pesanan; tanda_tangan RSA	Tinggi
Data akademik mahasiswa (NIM, fakultas)	Form registrasi mahasiswa; plaintext di DB	Sedang

Eksposur data rekening dan NIK pelaku UMKM memiliki implikasi keuangan langsung yang dapat menyebabkan kerugian finansial jika terjadi kebocoran. Oleh karena itu, enkripsi AES-256-GCM at-rest pada kolom-kolom ini adalah kontrol keamanan paling kritis dalam sistem.

2.2. Aliran Data Utama

Tabel 3 Aliran data utama sistem

Aliran	Deskripsi	Protokol
Browser to Backend	Request API dengan JWT Bearer token di Authorization header; body JSON request	HTTPS (Vercel TLS)
Backend tp PostgreSQL	SQLAlchemy ORM queries; kolom sensitif dienkripsi sebelum INSERT	TCP (private network)

Backend to Cloudinary	Upload gambar HTTPS produk/profil/bukti bayar via Cloudinary SDK
Backend to File System	AAA audit log ditulis ke Local I/O logs/aaa_accounting.log (JSON Lines)
Backend to DB (audit_logs)	Setiap HTTP request dicatat TCP (local) middleware ke tabel audit_logs
Token di Browser	JWT disimpan di - localStorage/sessionStorage atau memory; dikirim via Authorization: Bearer header

2.3. Aktor Ancaman

Tabel 4 Profil aktor ancaman

Aktor	Motivasi dan Kapabilitas	Kemungkinan
Penyerang data finansial	Mencuri data rekening/e-wallet UMKM untuk fraud keuangan	Sedang
Bot / Otomatis	Credential stuffing, brute force login akun mahasiswa atau UMKM	Tinggi
Pengguna internal nakal	Mahasiswa mengakses data pesanan UMKM lain; UMKM manipulasi pesanan	Sedang
Penyerang MitM	Intersepsi komunikasi jika HTTPS tidak dikonfigurasi (mitigated oleh Vercel TLS)	Rendah
Manipulator transaksi	Modifikasi data pesanan atau harga untuk mendapatkan harga lebih murah	Sedang
Penyerang XSS	Eksfiltrasi JWT dari localStorage via injeksi script	Sedang

3. Analisis STRIDE

3.1. Metodologi

Tabel 5 Kategori ancaman STRIDE

Huruf	Kategori	Deskripsi
S	Spoofing	Menyamar sebagai entitas lain yang sah
T	Tampering	Modifikasi data atau kode secara tidak sah
R	Repudiation	Menyangkal telah melakukan suatu tindakan
I	Information Disclosure	Eksposur informasi yang seharusnya rahasia
D	Denial of Service	Mengganggu ketersediaan layanan
E	Elevation of Privilege	Mendapatkan hak akses melebihi yang diberikan

3.2. Analisis per Komponen

3.2.1. Modul Autentikasi

File: backend/app/routers/auth.py and backend/app/core/security.py

Tabel 6 Analisis STRIDE modul autentikasi

ID	STRIDE	Ancaman	Risiko
AUTH-01	S	Credential stuffing terhadap akun mahasiswa dan UMKM; tidak ada rate limiting pada POST /api/auth/login	Tinggi
AUTH-02	D	Brute force password; endpoint login tidak dilindungi throttle; dapat membanjiri koneksi database	Tinggi
AUTH-03	S	Tidak ada mekanisme account lockout; percobaan login dapat dilakukan tanpa batas setelah login gagal berulang	Tinggi
AUTH-04	R	AAA log mencatat LOGIN_SUCCESS dan	Sedang

LOGIN_FAILED;
namun log disimpan lokal
(hilang di Vercel serverless) dan
di DB

AUTH-05	I	JWT tidak tersimpan di DB (stateless); revokasi token tidak mungkin tanpa restart server atau ganti SECRET_KEY	Sedang
---------	---	--	--------

3.2.2. Modul Guard dan Otorisasi (RBAC)

File: backend/app/core/security.py , fungsi get_current_user() dan require_role()

Tabel 7 Analisis STRIDE guard dan otorisasi

ID	STRIDE	Ancaman	Risiko
GUARD-01	E	Jika SECRET_KEY bocor dari environment variable, seluruh mekanisme JWT semua peran dapat dikompromikan sekaligus	Kritis
GUARD-02	E	Role dikodekan dalam JWT payload; tidak diverifikasi ulang ke database pada setiap request (hanya user_id yang dicek ke DB)	Sedang
GUARD-03	E	Tidak ada refresh token; access token berumur panjang memperbesar window of exposure jika token dicuri	Sedang
GUARD-04	S	Status akun (aktif/nonaktif) diverifikasi ke DB setiap request; user yang dinonaktifkan admin langsung ditolak (mitigated)	Rendah

3.2.3. Data Sensitif dan Enkripsi AES-256-GCM

File: backend/app/core/crypto.py dan backend/app/models/user.py (EncryptedString TypeDecorator)

Tabel 8 Analisis STRIDE data sensitif

ID	STRIDE	Ancaman	Risiko
DATA-01	I	Kolom no_telp, nik, nomor_rekening, nomor_ewallet dienkripsi AES-256-GCM via TypeDecorator ,DATA DILINDUNGI (mitigated)	Dimitigasi
DATA-02	T	GCM mode memberikan authenticated encryption; tampering pada ciphertext terdeteksi via InvalidTag exception (mitigated)	Dimitigasi
DATA-03	I	Kolom nama, email, NIM, fakultas disimpan plaintext; bukan data keuangan tetapi masih PII	Sedang
DATA-04	I	ENCRYPTION_KEY disimpan di environment variable Vercel; jika env var bocor, semua kolom terenkripsi dapat didekripsi	Tinggi

3.2.4. Tanda Tangan Digital RSA-2048 PSS

File: backend/app/auth/digital_signature.py

Tabel 9: Analisis STRIDE tanda tangan digital

ID	STRIDE	Ancaman	Risiko
SIG-01	R	Tanda tangan RSA-2048 PSS diterapkan saat pesanan selesai; non-repudiation terjamin ,tidak ada penolakan transaksi (mitigated)	Dimitigasi
SIG-02	T	Signature PSS dan SHA-256 memastikan integritas payload; tampering terdeteksi saat verifikasi (mitigated)	Dimitigasi
SIG-03	I	Private key RSA disimpan di file ./keys/private.pem;	Tinggi

		jika file system dapat diakses penyerang, private key bocor	
SIG-04	T	Verifikasi signature tidak dilakukan otomatis saat query pesanan; perlu dipanggil eksplisit oleh endpoint yang membutuhkan	Sedang

3.2.5. Upload File dan Media (Cloudinary)

File: backend/app/core/cloudinary_helper.py dan routers/pesanan.py

Tabel 10 Analisis STRIDE upload file

ID	STRIDE	Ancaman	Risiko
FILE-01	T	Upload bukti pembayaran (gambar) via Cloudinary; validasi MIME type tidak eksplisit di level backend FastAPI	Sedang
FILE-02	D	Tidak ada batas ukuran file yang dikonfigurasi di backend; upload file besar dapat memperparah penggunaan bandwidth dan storage	Sedang
FILE-03	I	URL gambar Cloudinary bersifat publik (tidak di-protect); siapa pun yang mengetahui URL dapat mengakses gambar	Rendah

4. Vulnerability Assessment Awal

4.1. Temuan Detail

4.1.1. [VA-01] Tidak Ada Rate Limiting pada Endpoint Login ,TINGGI

Kategori OWASP: A07:2021 ,Identification and Authentication Failures

Bukti teknis (routers/auth.py):

```
@router.post("/login", response_model=TokenResponse)
def login(request: Request,
```

```

form_data: OAuth2PasswordRequestForm = Depends(),
db: Session = Depends(get_db)):
# Tidak ada dekorator rate limiting / throttle
# Tidak ada pemeriksaan jumlah percobaan login sebelumnya

```

Dampak: Penyerang dapat melakukan credential stuffing atau brute force terhadap akun mahasiswa dan UMKM tanpa hambatan. Endpoint /api/auth/login tidak memiliki pembatasan request per IP maupun per akun.

Rekomendasi: Pasang SlowAPI (slowapi) atau middleware rate limiting khusus di FastAPI dengan konfigurasi maksimal 5 request/menit per IP pada endpoint login.

4.1.2. [VA-02] Tidak Ada Mekanisme Account Lockout ,TINGGI

Kategori OWASP: A07:2021 ,Identification and Authentication Failures

Bukti teknis (models/user.py):

```

class User(Base):
    user_id = Column(String, primary_key=True, ...)
    email = Column(String(100), unique=True, ...)
    password = Column(String(255), ...)
    status = Column(String(20), default="aktif")
# Tidak ada: failed_login_attempts, locked_until

```

Dampak: Tidak ada kolom failed_login_attempts atau locked_until pada tabel users. Akun dapat diserang brute force tanpa dibatasi jumlah percobaan gagal. Status akun (aktif/nonaktif) hanya dapat diubah manual oleh admin.

Rekomendasi: Tambahkan kolom failed_login_attempts INTEGER DEFAULT 0 dan locked_until TIMESTAMP NULL pada tabel users. Kunci akun sementara selama 15 menit setelah 5 kali login gagal; catat event LOCKED ke AAA log.

4.1.3. [VA-03] JWT Stateless ,Tidak Ada Revokasi Token Server-Side ,SEDANG

Kategori OWASP: A07:2021 ,Identification and Authentication Failures

Bukti teknis (core/security.py):

```

def get_current_user(token: str = Depends(oauth2_scheme),
db: Session = Depends(get_db)):
    payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
    user_id = payload.get("sub")

```

```
user = db.query(User).filter(User.user_id == user_id).first()
```

```
# Token tidak disimpan di DB; tidak ada pengecekan apakah token sudah di-revoke
```

Dampak: JWT bersifat stateless, token yang sudah dikeluarkan tetap valid hingga kadaluarsa meskipun pengguna telah 'logout'. Logout di frontend hanya menghapus token dari storage lokal, namun token yang sama masih dapat digunakan oleh penyerang jika berhasil mencurinya sebelum logout. Tidak ada cara untuk membatalkan token yang dicuri tanpa mengganti SECRET_KEY (yang akan memutus semua sesi aktif).

Rekomendasi: Simpan SHA-256 hash token yang aktif di database. Saat logout, hapus hash dari database. Guard memverifikasi hash token dari request cocok dengan hash di database. Alternatif: gunakan JWT expiry pendek (15 menit) dan refresh token berumur panjang yang disimpan di server.

4.1.4. [VA-04] CORS Mengizinkan Semua Method dan Header ,SEDANG

Kategori OWASP: A05:2021 ,Security Misconfiguration

Bukti teknis (main.py):

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=ALLOWED_ORIGINS, # ✓ origin dikontrol
    allow_credentials=True,
    allow_methods=["*"],           # ✗ semua HTTP method diizinkan
    allow_headers=["*"],           # ✗ semua header diizinkan
)
```

Dampak: Meskipun allow_origins sudah direstriksi ke whitelist domain spesifik, allow_methods=["*"] dan allow_headers=["*"] memberikan permukaan serangan yang lebih luas dari yang diperlukan. Method seperti DELETE atau PATCH dapat dipanggil dari origin yang diizinkan tanpa pembatasan method eksplisit.

Rekomendasi: Batasi allow_methods ke method yang benar-benar digunakan: ["GET", "POST", "PUT", "DELETE", "OPTIONS"]. Batasi allow_headers ke header yang diperlukan: ["Authorization", "Content-Type", "Accept"].

4.1.5. [VA-05] Tidak Ada Refresh Token ,SEDANG

Kategori OWASP: A07:2021 ,Identification and Authentication Failures

Bukti teknis (core/security.py):

```
def create_access_token(data: dict, expires_delta=None) -> str:
    expire = datetime.now(timezone.utc) dan (
        expires_delta or timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    )
    # Hanya satu jenis token; tidak ada access dan refresh token separation
```

Dampak: Sistem hanya menggunakan satu jenis token (access token) dengan durasi dikonfigurasi di ACCESS_TOKEN_EXPIRE_MINUTES. Jika nilai ini besar (misal 24 jam), window of exposure token yang dicuri sangat besar. Jika kecil (misal 15 menit), pengguna harus login ulang terlalu sering.

Rekomendasi: Implementasikan dua jenis token: access token berumur pendek (15-60 menit) dan refresh token berumur panjang (7-30 hari) yang disimpan di HttpOnly cookie. Endpoint /api/auth/refresh memperbarui access token menggunakan refresh token.

4.1.6. [VA-06] Validasi MIME Type Upload Gambar Tidak Eksplisit ,SEDANG

Kategori OWASP: A04:2021 ,Insecure Design

Bukti teknis: Upload gambar menggunakan Cloudinary SDK. Validasi MIME type dilakukan implisit oleh Cloudinary, namun tidak ada pemeriksaan eksplisit di level backend FastAPI sebelum file dikirim ke Cloudinary. File dengan ekstensi .jpg namun konten berbeda tidak akan ditolak di level aplikasi.

Rekomendasi: Tambahkan validasi MIME type di backend sebelum upload ke Cloudinary: periksa magic bytes file (python-magic) dan batasi ekstensi yang diizinkan ke ['.jpg', '.jpeg', '.png', '.webp']. Tambahkan batas ukuran file maksimal (misal 5 MB) di level middleware FastAPI.

4.1.7. [VA-07] Inkonsistensi Penanganan Error ,RENDAH

Kategori OWASP: A09:2021 ,Security Logging and Monitoring Failures

Bukti teknis (routers/auth.py):

```
# Login ,pesan generik (baik):
raise HTTPException(status_code=401, detail="Email atau password salah")

# Register ,pesan spesifik:
raise HTTPException(status_code=400, detail="Email sudah terdaftar")
```

Dampak: Pesan 'Email sudah terdaftar' pada endpoint register memungkinkan penyerang melakukan user enumeration ,mengetahui email mana yang sudah terdaftar di sistem. Ini dapat dimanfaatkan untuk targeted phishing.

Rekomendasi: Pertimbangkan pesan generik pada register: 'Registrasi gagal, periksa data Anda' alih-alih mengekspos status email. Atau implementasikan verified email flow (kode OTP ke email) yang secara natural menyembunyikan enumerasi.

5. Matriks Risiko

Tabel 11: Matriks risiko semua temuan *AUTH-01 dikategorikan Kritis berdasarkan kemungkinan yang sangat tinggi

ID	Kemungkinan	Dampak	Skor	Level	Judul
GUARD-01	2	5	10	Tinggi	SECRET_KEY bocor to seluruh JWT dikompromikan
AUTH-01	5	4	20	Kritis*	Credential stuffing (tanpa rate limiting)
AUTH-02	4	4	16	Tinggi	Brute force login (tanpa rate limiting)
AUTH-03	4	4	16	Tinggi	Tidak ada account lockout
DATA-04	2	5	10	Tinggi	ENCRYPTION_KEY bocor dari env var
SIG-03	2	4	8	Sedang	RSA private key bocor dari file system
AUTH-05	3	3	9	Sedang	JWT stateless ,tidak ada revokasi

GUARD-03	3	3	9	Sedang	Tidak ada refresh token
VA-04	3	3	9	Sedang	CORS allow-all methods dan headers
DATA-03	3	3	9	Sedang	NIM/nama/email disimpan plaintext
FILE-01	2	4	8	Sedang	Upload gambar tanpa validasi MIME
FILE-02	2	3	6	Sedang	Tidak ada batas ukuran file upload
SIG-04	2	3	6	Sedang	Verifikasi signature tidak otomatis
VA-07	3	2	6	Rendah	User enumeration via pesan error register
FILE-03	2	2	4	Rendah	URL Cloudinary bersifat publik
DATA-01	—	—	—	Dimitigasi	AES-256-GCM enkripsi PII (sudah diimplementasi)
SIG-01				Dimitigasi	RSA-2048 PSS non-repudiation (sudah diimplementasi)

5.1. Prioritas Remediasi

Tabel 12: Prioritas remediasi

Prioritas	ID	Tindakan	Effort
P1	AUTH-01/02	Pasang SlowAPI rate limiting pada POST /api/auth/login (maks 5 req/menit/IP)	Rendah
P1	AUTH-03	Implementasikan account lockout: kolom failed_login_attempts dan locked_until di User model	Sedang
P2	AUTH-05	Simpan SHA-256 hash token di DB; hapus saat logout (token revocation)	Sedang
P2	GUARD-03	Implementasikan access dan refresh token dengan expiry eksplisit	Tinggi
P2	VA-04	Batasi CORS allow_methods dan allow_headers ke daftar minimal yang diperlukan	Rendah
P3	FILE-01	Tambahkan validasi MIME type (magic bytes) sebelum upload ke Cloudinary	Sedang

P3	FILE-02	Konfigurasi batas ukuran file di FastAPI middleware (maks 5 MB)	Rendah
P3	VA-07	Standarisasi pesan error register agar tidak mengekspos enumerasi email	Rendah
P4	GUARD-01	Pastikan SECRET_KEY disimpan aman di Vercel env; rotasi berkala	Rendah
P4	DATA-04	Pastikan ENCRYPTION_KEY tidak pernah di-commit ke git; gunakan secret manager	Rendah
P4	SIG-03	Pindahkan private key RSA dari file ke Vercel env var (base64-encoded)	Sedang

6. Rekomendasi Implementasi

6.1. Jangka Pendek

6.1.1. Rate Limiting dengan SlowAPI

```
# requirements.txt: tambahkan slowapi
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded
```



```

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)

```

```

# Di routers/auth.py:
@router.post("/login")
@limiter.limit("5/minute") # ← tambahkan ini
def login(request: Request, ...):

```

6.1.2. Account Lockout

```

# Tambahkan ke class User (models/user.py):
failed_login_attempts = Column(Integer, default=0)
locked_until          = Column(DateTime, nullable=True)

# Di routers/auth.py ,setelah login gagal:
user.failed_login_attempts += 1
if user.failed_login_attempts >= 5:
    user.locked_until = datetime.now(timezone.utc) + timedelta(minutes=15)
    log_accounting(user_id=user.id, action="LOCKED", ip=ip)
db.commit()

```

6.2. Jangka Menengah

6.2.1. Token Revocation via Hash di Database

```

import hashlib

# Saat login ,simpan hash token (bukan token asli):
token_hash = hashlib.sha256(token.encode()).hexdigest()
user.token_hash = token_hash # kolom baru di User model

# Di get_current_user() ,verifikasi hash:
incoming_hash = hashlib.sha256(token.encode()).hexdigest()
user = db.query(User).filter(
    User.user_id == user_id,
    User.token_hash == incoming_hash # ← cek revocation
).first()

# Saat logout ,hapus hash:

```

```
user.token_hash = None
db.commit()
```

6.2.2. CORS Lebih Ketat

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=ALLOWED_ORIGINS,
    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "DELETE", "OPTIONS"],
    allow_headers=["Authorization", "Content-Type", "Accept"],
)
```

7. Kesimpulan

IPB Food Hub & UMKM memiliki fondasi keamanan kriptografi yang kuat, AES-256-GCM untuk enkripsi data PII sensitif at-rest, RSA-2048 PSS untuk non-repudiation transaksi, bcrypt untuk password hashing, RBAC berbasis JWT dengan verifikasi status akun ke database, serta sistem audit log komprehensif dua lapisan (file dan database) yang dapat dimonitor melalui Security Dashboard real-time.

Namun, terdapat celah operasional yang signifikan pada lapisan autentikasi: tidak adanya rate limiting dan account lockout membuka pintu bagi serangan brute force dan credential stuffing yang merupakan vektor serangan paling umum pada aplikasi web. JWT yang bersifat stateless juga berarti token yang dicuri tidak dapat dibatalkan tanpa mengganti SECRET_KEY ,risiko yang dapat dimitigasi dengan implementasi token revocation berbasis hash di database.

Prioritas remediasi utama seharusnya mencerminkan profil ancaman nyata platform ini: implementasikan rate limiting dan account lockout terlebih dahulu (jangka pendek, effort rendah, dampak besar), kemudian lanjutkan dengan token revocation dan refresh token (jangka menengah). Keamanan kriptografi yang sudah ada memberikan differensiasi positif terhadap platform serupa yang tidak mengimplementasikan enkripsi at-rest maupun digital signature.